

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 3 Practical No: 1

Roll no: 63 PRN No: 12410410 Name of Student: **Pratham Shelke**

Subject Name : Deep Learning

Problem Statement : Install and configure TensorFlow/Keras in Google Colab. Perform data preprocessing, normalization, train-test split, and visualization on a sample dataset.

Algorithm :

1. Start Google Collab and create a new notebook.
2. Import the required libraries such as TensorFlow, NumPy, Pandas, Matplotlib, and Scikit-learn.
3. Load a sample dataset (Iris Dataset).
4. Display the first few records of the dataset.
5. Check for missing values and dataset information.
6. Separate features (X) and target labels (y).
7. Normalize the feature values using Min-Max Scaling.
8. Split the dataset into training and testing sets using an 80:20 ratio.
9. Visualize the dataset using scatter plots and histograms.
10. Import TensorFlow and Keras to verify successful installation.

11. Display TensorFlow and Keras versions.
12. Print the shapes of training and testing datasets.
13. Observe and analyze the output.

Code :

```
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np

print(tf.__version__)

fashion_mnist = tf.keras.datasets.fashion_mnist

(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 — 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 — 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 — 0s 1us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 — 0s 0us/step
```

```
print("Training images shape:", train_images.shape)
print("Training labels shape:", train_labels.shape)

print("Testing images shape:", test_images.shape)
print("Testing labels shape:", test_labels.shape)
```

[6]

```
... Training images shape: (60000, 28, 28)
Training labels shape: (60000,)
Testing images shape: (10000, 28, 28)
Testing labels shape: (10000,)
```

```
class_names = [
    'T-shirt/top',
    'Trouser',
    'Pullover',
    'Dress',
    'Coat',
    'Sandal',
    'Shirt',
    'Sneaker',
    'Bag',
    'Ankle boot'
]
```

[7]

```
print("First image label:", train_labels[0])
print("Class name:", class_names[train_labels[0]])
```

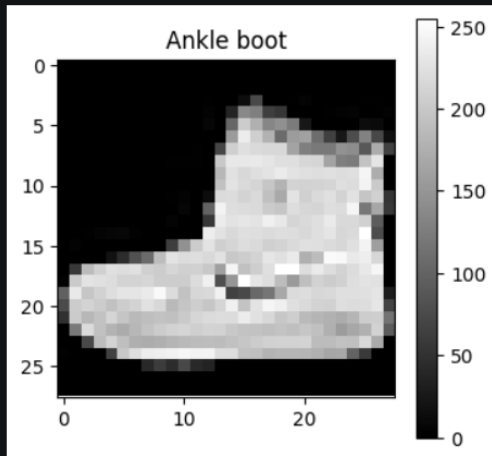
[8]

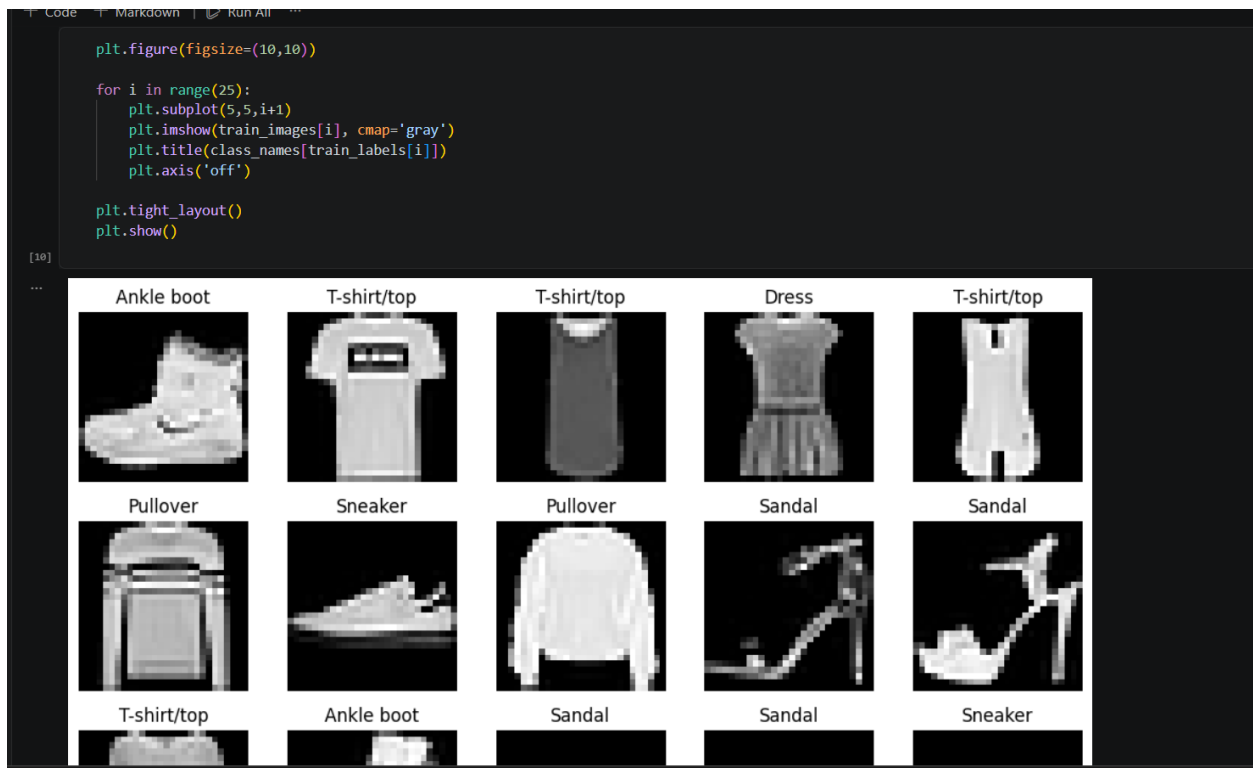
```
... First image label: 9
Class name: Ankle boot
```

```
plt.figure(figsize=(4,4))
plt.imshow(train_images[0], cmap='gray')
plt.title(class_names[train_labels[0]])
plt.colorbar()
plt.show()
```

[9]

...





```
train_images = train_images / 255.0
test_images = test_images / 255.0
```

[11] Python

```
print("Minimum pixel value:", train_images.min())
print("Maximum pixel value:", train_images.max())
```

[12] Python

... Minimum pixel value: 0.0
Maximum pixel value: 1.0

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
```

[13] Python

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential model super().__init__(**kwargs)

```
model.summary()
```

[14] Python

... Model: "sequential"

```
... Model: "sequential"
```

```
...
```

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290

```
...
```

```
Total params: 101,770 (397.54 KB)
```

```
...
```

```
Trainable params: 101,770 (397.54 KB)
```

```
...
```

```
Non-trainable params: 0 (0.00 B)
```

```
model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```

```
[15]
```

```
history = model.fit(  
    train_images,  
    train_labels,  
    epochs=5  
)
```

```
[17]
```

```
... Epoch 1/5  
1875/1875 ————— 7s 4ms/step - accuracy: 0.8707 - loss: 0.3540  
Epoch 2/5  
1875/1875 ————— 10s 3ms/step - accuracy: 0.8805 - loss: 0.3267  
Epoch 3/5  
1875/1875 ————— 7s 4ms/step - accuracy: 0.8876 - loss: 0.3047  
Epoch 4/5  
1875/1875 ————— 6s 3ms/step - accuracy: 0.8936 - loss: 0.2867  
Epoch 5/5  
1875/1875 ————— 7s 4ms/step - accuracy: 0.8983 - loss: 0.2743
```

```
test_loss, test_accuracy = model.evaluate(test_images, test_labels)  
  
print("Test Loss:", test_loss)  
print("Test Accuracy:", test_accuracy)
```

```
[18]
```

```
... 313/313 ————— 1s 2ms/step - accuracy: 0.8786 - loss: 0.3432  
Test Loss: 0.3432213068008423  
Test Accuracy: 0.878600001335144
```

```
predictions = model.predict(test_images)
```

```
[19]
```

```
[19] predictions = model.predict(test_images)

... 313/313 ————— 0s 1ms/step

import numpy as np

predicted_label = np.argmax(predictions[0])

print("Predicted Label :", predicted_label)
print("Predicted Class :", class_names[predicted_label])

print("Actual Label :", test_labels[0])
print("Actual Class :", class_names[test_labels[0]])

[20] Predicted Label : 9
Predicted Class : Ankle boot
Actual Label : 9
Actual Class : Ankle boot

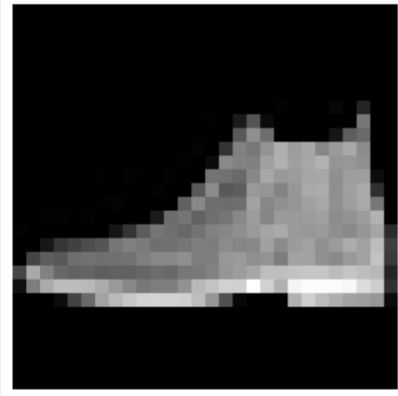
plt.figure(figsize=(4,4))
plt.imshow(test_images[0], cmap='gray')
plt.title(f"Predicted: {class_names[predicted_label]}")
plt.axis('off')
plt.show()

[21]
```

```
plt.figure(figsize=(4,4))
plt.imshow(test_images[0], cmap='gray')
plt.title(f"Predicted: {class_names[predicted_label]}")
plt.axis('off')
plt.show()

[21]

... Predicted: Ankle boot

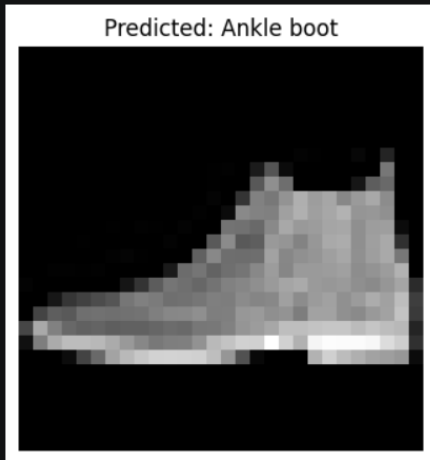


model.save("fashion_mnist_model.keras")

[22]
```

Output :

```
[21] plt.figure(figsize=(4,4))
plt.imshow(test_images[0], cmap='gray')
plt.title(f"Predicted: {class_names[predicted_label]}")
plt.axis('off')
plt.show()
```



```
[22] model.save("fashion_mnist_model.keras")
```

```
test_loss, test_accuracy = model.evaluate(test_images, test_labels)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

[18]

... 313/313 ————— 1s 2ms/step - accuracy: 0.8786 - loss: 0.3432
Test Loss: 0.3432213068008423
Test Accuracy: 0.878600001335144

```
predictions = model.predict(test_images)
```

[19]

... 313/313 ————— 0s 1ms/step

```
import numpy as np

predicted_label = np.argmax(predictions[0])

print("Predicted Label :", predicted_label)
print("Predicted Class :", class_names[predicted_label])

print("Actual Label :", test_labels[0])
print("Actual Class :", class_names[test_labels[0]])
```

[20]

... Predicted Label : 9
Predicted Class : Ankle boot
Actual Label : 9
Actual Class : Ankle boot

```
train_images = train_images / 255.0
test_images = test_images / 255.0

print("Minimum pixel value:", train_images.min())
print("Maximum pixel value:", train_images.max())

... Minimum pixel value: 0.0
Maximum pixel value: 1.0

model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
```



```

# Import required libraries: TensorFlow/Keras for building and
training the neural network,
# Matplotlib for visualizing images, and NumPy for numerical
operations
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np

# Print the installed TensorFlow version to confirm the environment
setup
print(tf.__version__)

2.20.0

# Load a reference to the built-in Fashion MNIST dataset module from
Keras
fashion_mnist = tf.keras.datasets.fashion_mnist

# Download the Fashion MNIST dataset and split it into training and
testing sets
# (images and their corresponding labels)
(train_images, train_labels), (test_images, test_labels) =
fashion_mnist.load_data()

Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 _____ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 _____ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 _____ 0s 1us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 _____ 0s 0us/step

# Print the shapes of the training and testing image/label arrays to
verify the data
# (60,000 training images, 10,000 testing images, each 28x28 pixels)
print("Training images shape:", train_images.shape)
print("Training labels shape:", train_labels.shape)

print("Testing images shape:", test_images.shape)
print("Testing labels shape:", test_labels.shape)

Training images shape: (60000, 28, 28)
Training labels shape: (60000,)
Testing images shape: (10000, 28, 28)
Testing labels shape: (10000,)

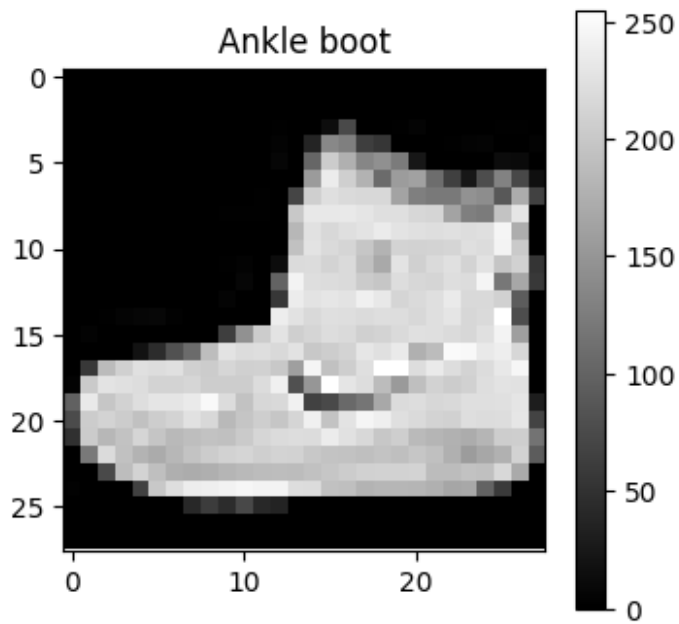
```

```
# Define human-readable class names corresponding to the 10 numeric
label classes (0-9)
# in the Fashion MNIST dataset
class_names = [
    'T-shirt/top',
    'Trouser',
    'Pullover',
    'Dress',
    'Coat',
    'Sandal',
    'Shirt',
    'Sneaker',
    'Bag',
    'Ankle boot'
]

# Display the numeric label and corresponding class name for the first
training image
# as a sanity check
print("First image label:", train_labels[0])
print("Class name:", class_names[train_labels[0]])

First image label: 9
Class name: Ankle boot

# Visualize the first training image as a grayscale plot with its
class name as the title
# and a colorbar showing pixel intensity values
plt.figure(figsize=(4,4))
plt.imshow(train_images[0], cmap='gray')
plt.title(class_names[train_labels[0]])
plt.colorbar()
plt.show()
```



```
# Plot a 5x5 grid of the first 25 training images along with their  
class names  
# to get a broader look at the dataset  
plt.figure(figsize=(10,10))  
  
for i in range(25):  
    plt.subplot(5,5,i+1)  
    plt.imshow(train_images[i], cmap='gray')  
    plt.title(class_names[train_labels[i]])  
    plt.axis('off')  
  
plt.tight_layout()  
plt.show()
```



```
# Normalize pixel values from the original [0, 255] range to [0, 1]
# by dividing by 255.0 (helps the model train faster and more
effectively)
```

```
train_images = train_images / 255.0
test_images = test_images / 255.0
```

```
# Verify normalization by printing the minimum and maximum pixel
values
```

```
# (should now be 0.0 and 1.0)
```

```
print("Minimum pixel value:", train_images.min())
```

```
print("Maximum pixel value:", train_images.max())
```

```
Minimum pixel value: 0.0
Maximum pixel value: 1.0
```

```
# Build a simple feedforward neural network using Keras Sequential
API:
```

```
# - Flatten layer converts each 28x28 image into a 1D array of 784
pixels
```

```
# - Dense hidden layer with 128 neurons and ReLU activation
```

```
# - Dense output layer with 10 neurons (one per class) and softmax
activation
```

```
# to output class probabilities
```

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/reshaping/
flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
```

```
# Print a summary of the model architecture, including layer types,
# output shapes, and number of trainable parameters
```

```
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	
Param #		
flatten (Flatten)	(None, 784)	
0		
dense (Dense)	(None, 128)	
100,480		
dense_1 (Dense)	(None, 10)	
1,290		

```
Total params: 101,770 (397.54 KB)
```

```
Trainable params: 101,770 (397.54 KB)
```

Non-trainable params: 0 (0.00 B)

```
# Compile the model by specifying the optimizer (Adam), loss function  
# (sparse categorical crossentropy, suitable for integer-labeled  
multi-class problems),  
# and the metric to track (accuracy)
```

```
model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```

```
# Train the model on the training images and labels for 5 epochs
```

```
history = model.fit(  
    train_images,  
    train_labels,  
    epochs=5  
)
```

```
Epoch 1/5  
1875/1875 ————— 6s 3ms/step - accuracy: 0.8237 - loss:  
0.5038
```

```
Epoch 2/5  
1875/1875 ————— 5s 3ms/step - accuracy: 0.8636 - loss:  
0.3781
```

```
Epoch 3/5  
1875/1875 ————— 5s 2ms/step - accuracy: 0.8770 - loss:  
0.3385
```

```
Epoch 4/5  
1875/1875 ————— 5s 2ms/step - accuracy: 0.8854 - loss:  
0.3126
```

```
Epoch 5/5  
1875/1875 ————— 6s 3ms/step - accuracy: 0.8920 - loss:  
0.2951
```

```
# Evaluate the trained model on the unseen test dataset and print the  
# resulting test loss and test accuracy
```

```
test_loss, test_accuracy = model.evaluate(test_images, test_labels)
```

```
print("Test Loss:", test_loss)
```

```
print("Test Accuracy:", test_accuracy)
```

```
313/313 ————— 1s 2ms/step - accuracy: 0.8793 - loss:  
0.3380
```

```
Test Loss: 0.3380337953567505
```

```
Test Accuracy: 0.8792999982833862
```

```
# Generate predictions (class probability distributions) for all test  
images
```

```
predictions = model.predict(test_images)
```

313/313 ————— 1s 2ms/step

```
# Get the predicted class for the first test image by taking the index  
# with the highest probability, then compare it against the actual  
label
```

```
import numpy as np
```

```
predicted_label = np.argmax(predictions[0])
```

```
print("Predicted Label :", predicted_label)
```

```
print("Predicted Class :", class_names[predicted_label])
```

```
print("Actual Label :", test_labels[0])
```

```
print("Actual Class :", class_names[test_labels[0]])
```

```
Predicted Label : 9
```

```
Predicted Class : Ankle boot
```

```
Actual Label : 9
```

```
Actual Class : Ankle boot
```

```
# Visualize the first test image alongside the model's predicted class  
label
```

```
plt.figure(figsize=(4,4))
```

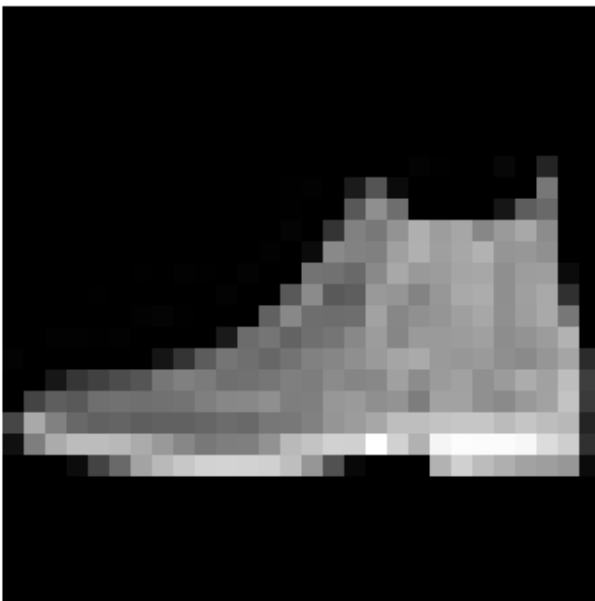
```
plt.imshow(test_images[0], cmap='gray')
```

```
plt.title(f"Predicted: {class_names[predicted_label]}")
```

```
plt.axis('off')
```

```
plt.show()
```

Predicted: Ankle boot



```
# Save the trained model to disk in Keras format for later reuse  
model.save("fashion_mnist_model.keras")
```